

Tokenization Evaluation and Character Normalization

A comparative preprocessing study on informal Twitter/X text

Course	Information Retrieval
Dataset	Tesla Twitter Corpus (Tesla.csv)
Tasks	Task 1: Tokenization Performance Evaluation Task 2: Repeated Character Normalization
Repository	GitHub Repository

Executive Summary

This report presents the implementation and evaluation of two Information Retrieval preprocessing tasks on Twitter/X text. **Task 1** compares three tokenization approaches - NLTK TweetTokenizer, spaCy (en_core_web_sm), and NLTK word_tokenize - on their ability to preserve hashtags, user mentions, URLs, and emojis. **Task 2** develops a frequency-based normalization pipeline for elongated character sequences such as “soooo” and “yasss”, while protecting structured entities including hashtags, mentions, URLs, and cashtags.

The evaluation uses the Tesla Twitter Corpus (Tesla.csv). Task 1 is based on 50 English tweets selected for high coverage of the target social-media elements. Task 2 evaluates 100 tweets containing character elongation. The resulting measurements, case studies, and failure analysis are reported directly from the implemented preprocessing pipeline.

1. Introduction

Social-media text presents a different preprocessing problem from conventional prose. Hashtags, handles, URLs, emojis, and informal spellings often carry useful retrieval information, but general-purpose tokenizers can split them into fragments. Character elongation introduces a second problem: expressive forms such as “sooooo” and “goodddd” create out-of-vocabulary variants that may not match standard query terms.

1.1 Objectives

- Compare domain-specific and general-purpose tokenizers on Twitter-native elements.
- Measure hashtag, mention, URL, and emoji preservation using exact token matching.
- Measure the total number of tokens generated by each tokenizer.
- Design a conservative normalization method for repeated alphabetic characters.
- Protect structured social-media entities during normalization and examine representative success and failure cases.

1.2 Dataset and Repository

Both tasks use the Tesla Twitter Corpus (Tesla.csv). The implementation, selection scripts, evaluation scripts, and output files are maintained in the project repository: github.com/tony19053000/Information-Retrieval-Assignment

2. Task 1 - Tokenization Performance Evaluation

2.1 Experimental Setup

Fifty English tweets were selected from Tesla.csv using the deterministic selection procedure in src/select_ds.py. The subset was chosen to provide substantial coverage of hashtags, mentions, URLs, and emojis. The same 50 posts were processed by all three tokenizers so that the comparison used an identical input set.

Tokenizer	Role in comparison	Configuration / description
NLTK TweetTokenizer	Domain-specific rule-based tokenizer	preserve_case=True; reduce_len=False; strip_handles=False
spaCy (en_core_web_sm)	General-purpose NLP tokenizer	Default English pipeline tokenization
NLTK word_tokenize	Treebank/Punkt-style word tokenizer	Conventional word tokenization baseline

Table 1. Tokenizers evaluated in Task 1.

2.2 Ground Truth and Evaluation Criterion

Ground-truth social-media elements were extracted from the raw tweet text before tokenization. Hashtags, mentions, and URLs were identified using regular expressions, while emojis were parsed using the Python emoji package. The extracted targets were stored in data/task1_ground_truth.csv.

Element	Extraction method	Example
Hashtag	(?<!\w)#[A-Za-z0-9_]+	#Tesla
User mention	(?<!\w)@[A-Za-z0-9_]+	@elonmusk
URL	https?:\/\/[^\s]+ www\.[^\s]+	https://example.com
Emoji	Python emoji package	Unicode emoji sequence

Table 2. Ground-truth extraction rules.

An item was counted as preserved only when the complete ground-truth string appeared as a single token in the tokenizer output. This exact-match rule is intentionally strict: for example, #Tesla is preserved as one token, but not when it is split into “#” and “Tesla”.

2.3 Procedure

1. Select 50 English tweets from the corpus using the deterministic selection script.
2. Extract hashtags, mentions, URLs, and emojis from the original text to form the ground truth.
3. Apply TweetTokenizer, spaCy, and word_tokenize to the same set of posts.
4. Compare each output against the exact-match ground truth for each target element.
5. Record preservation percentages, error-post counts, total token counts, and average tokens per post.

3. Task 1 Results and Discussion

Metric	TweetTokenizer	spaCy	word_tokenize
Hashtag preservation	100.00%	0.00%	0.00%
Mention preservation	100.00%	97.78%	0.00%
URL preservation	100.00%	100.00%	0.00%
Emoji preservation	95.35%	95.35%	34.88%

Table 3. Preservation rates across 50 tweets.

Error-post count	TweetTokenizer	spaCy	word_tokenize
Hashtag errors	0	23	23
Mention errors	0	3	44
URL errors	0	0	24
Emoji errors	2	2	13

Table 4. Number of posts containing preservation errors.

Token metric	TweetTokenizer	spaCy	word_tokenize
Total tokens generated	1,814	2,013	2,151
Average tokens per post	36.28	40.26	43.02

Table 5. Token-generation totals.

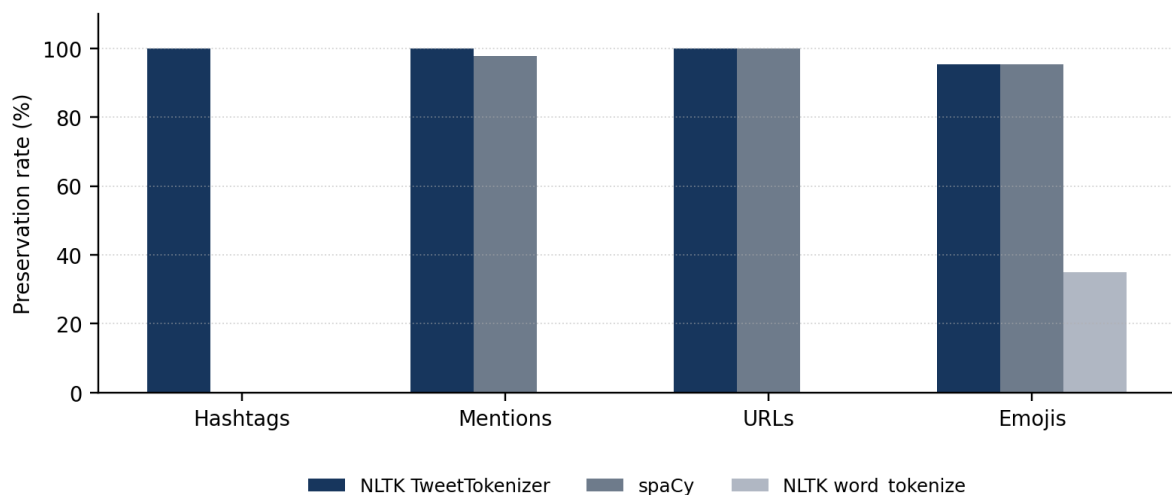


Figure 1. Preservation rate by social-media element.

3.1 Interpretation of the Results

Hashtags. TweetTokenizer preserved all hashtags. spaCy and word_tokenize separated the leading # from the following term, so they scored 0% under the exact-token criterion.

Mentions and URLs. TweetTokenizer preserved both categories completely. spaCy preserved all URLs and 97.78% of mentions, with the remaining mention errors arising in cases where handles were split. word_tokenize fragmented both kinds of structure and scored 0% for preservation.

Emojis. TweetTokenizer and spaCy both achieved 95.35% preservation, with two emoji-error posts each. word_tokenize reached 34.88% preservation and recorded 13 emoji-error posts.

4. Tokenization Findings

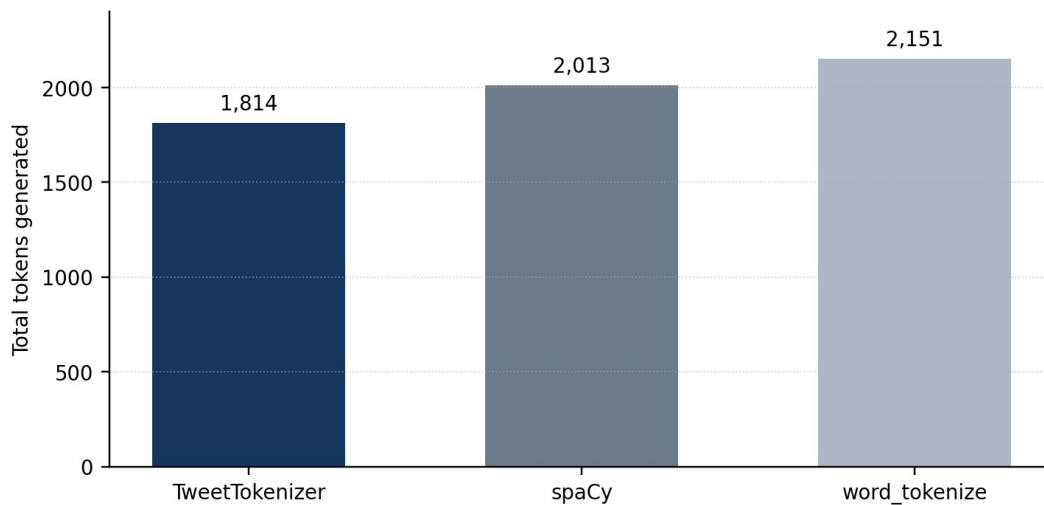


Figure 2. Total tokens generated by each tokenizer.

4.1 Token Count and Vocabulary Fragmentation

NLTK `word_tokenize` generated 2,151 tokens, compared with 1,814 from TweetTokenizer. This is an 18.5% over-tokenization overhead relative to TweetTokenizer. In this experiment, the larger token count does not indicate better tokenization; much of the increase comes from fragmenting URLs, handles, and other structures into smaller pieces, which can increase inverted-index payload size and vocabulary fragmentation.

4.2 Comparative Strengths and Weaknesses

NLTK TweetTokenizer. It achieved complete preservation of hashtags, mentions, and URLs and 95.35% emoji preservation, while producing the fewest tokens of the three approaches. The exact-match evaluation recorded two emoji-error posts.

spaCy. It preserved all URLs, most mentions, and 95.35% of emojis. Its default tokenizer, however, split hashtags and a small number of mentions. It remains useful when tokenization is required as part of a broader NLP pipeline.

NLTK `word_tokenize`. It produced predictable conventional word-tokenization output but performed poorly on Twitter-native structures. The tokenizer split hashtags, mentions, and URLs and preserved only 34.88% of emojis, resulting in the greatest degree of fragmentation.

4.3 Task 1 Conclusion

For the evaluated Twitter/X features, NLTK TweetTokenizer provides the strongest overall preservation while also producing the smallest token count. spaCy performs well on URLs, mentions, and emojis but requires additional handling for hashtags. NLTK `word_tokenize` is the least suitable of the three for this social-media preprocessing setting because it systematically fragments the structures being evaluated.

5. Task 2 - Repeated Character Normalization

5.1 Problem Formulation

Twitter/X users frequently repeat characters for emphasis, producing forms such as “sooooo”, “goodddd”, and “yasss”. In an Information Retrieval system, these variants can increase the number of out-of-vocabulary terms and reduce matching with standard index terms. The goal of Task 2 is therefore to detect excessive alphabetic repetition, recover a plausible standard form, and protect structured entities that should not be rewritten.

5.2 Requirements

- Detect runs of three or more identical alphabetic characters.
- Generate normalization candidates that can recover standard lexical forms.
- Preserve hashtags, mentions, URLs, and cashtags exactly during normalization.
- Retain the original casing convention where applicable.

5.3 Repetition Detection

The repetition detector uses the following regular expression:

```
([A-Za-z])\1{2,}
```

The capturing group stores one alphabetic character, while the backreference requires the same character to appear at least two additional times. The threshold of three or more repetitions avoids treating ordinary double letters as noise.

5.4 Protection Rule

Before normalization, URLs, hashtags, mentions, and cashtags are replaced with temporary placeholders. This prevents repeated characters inside an identifier from being interpreted as spelling noise.

```
https?:\/\/[^\s]+ | www\.[^\s]+ | (?<!\w)#[A-Za-z0-9_]+ | (?<!\w)@[A-Za-z0-9_]+ | (?<!\w)\$[A-Za-z0-9_]+
```

5.5 Candidate Selection and Casing Restoration

For each detected repetition, the implementation generates one-repeat and two-repeat candidates and compares them using Zipf word-frequency scores from the wordfreq package. The higher-scoring candidate is selected as the normalized form. After this step, the original casing convention is restored and the protected entities are placed back into the tweet unchanged.

6. Normalization Pipeline and Evaluation

6.1 Pipeline

1	Protection masking	Replace URLs, #hashtags, @mentions, and \$cashtags with temporary protected placeholders.
2	Pattern detection	Match words containing character runs identified by $([A-Za-z])\{2,\}$.
3	Candidate scoring	Generate one-repeat and two-repeat candidates and select the candidate with the highest Zipf score using wordfreq.
4	Casing restoration and unmasking	Restore UPPERCASE / Capitalized formatting and replace placeholders with the original protected entities.

Table 6. Four-stage normalization pipeline implemented in `src/task2_normalize.py`.

6.2 Qualitative Evaluation

The normalization stage was evaluated on 100 sample tweets containing character elongation. The examples below illustrate both lexical correction and entity protection.

Post	Original and normalized text	Assessment
1	Original: Whip a Tesla Whip a fucking teslaaaaaaa Normalized: Whip a Tesla Whip a fucking tesla	Correct
4	Original: ...makes bad unsafe cars and is waaaaay overvalued... Normalized: ...and is way overvalued...	Correct
5	Original: ... \$TSLA \$QQQ \$AAPL \$NVDA \$AMZN \$SQQQ \$TQQQ ... Normalized: financial symbols unchanged	Protected cashtags
6	Original: Probably Betcoin got sideswiped by this narcissist as well. Normalized: Probably Betcoin got sideswiped by this narcissist as well.	Correct
9	Original: @MichelleBekke11 ... hahahahaaaaa Normalized: @MichelleBekke11 ... hahahahaha	Correct
10	Original: @elonmusk Liar! Compared to Tesla in history, you are noooooothing. Normalized: @elonmusk Liar! Compared to Tesla in history, you are nothing.	Correct

Table 7. Representative normalization cases from the 100-tweet evaluation subset.

6.3 Observations

The examples show that the pipeline can reduce elongated spelling while leaving protected identifiers unchanged. For each detected repetition, the implementation compares one-repeat and two-repeat candidates using Zipf word-frequency scores before selecting the normalized form. The protection stage ensures that financial cashtags and other structured social-media identifiers pass through the normalization process unchanged.

7. Conclusion and Recommendations

7.1 Overall Findings

The two tasks address complementary preprocessing problems in social-media Information Retrieval. Task 1 shows that tokenizer choice directly affects whether Twitter-native structures survive preprocessing. NLTK TweetTokenizer provided the strongest overall preservation on the evaluated features and also generated fewer tokens than the other two approaches. Task 2 shows that repeated-character normalization can reduce informal lexical variation when it is combined with protected-entity masking and frequency-based candidate selection.

7.2 System Recommendations

1. Use NLTK TweetTokenizer as the primary tokenizer for this Twitter/X preprocessing setting because it preserves domain-specific structures with minimal fragmentation.
2. Apply repeated-character normalization before indexing when the application benefits from matching expressive spellings to standard forms.
3. Retain masking for URLs, hashtags, mentions, and cashtags so that identifiers and financial symbols are not changed by the normalization rule.
4. Keep the evaluation outputs and scripts alongside the dataset so reported figures can be reproduced from the implementation.

7.3 Repository

The project source code, dataset scripts, test pipelines, and output files are available at: <https://github.com/tony19053000/Information-Retrieval-Assignment>

8. AI Usage Statement

AI coding tools (Antigravity / Gemini) were used for environment-setup script drafting, regular-expression optimization - particularly for entity-protection masks covering hashtags, mentions, and cashtags - and report formatting. All experimental datasets (data/task1_posts.csv and data/task2_posts.csv), ground-truth extraction, tokenizer performance calculations, and normalization error analyses were executed and verified locally using the implemented Python scripts.